

MYSQL STORED PROCEDURES TASK 1:

Create a procedure that displays all employees:

```
C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql -u root -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database |
+-----+
| bookflow1db |
| bookflow2db |
| bookflow3_db |
| bookflow_db |
| bookflowdb |
| employeeedb1 |
| employeeedb2 |
| employeeedb3 |
| employeeedb4 |
| employeesdb |
| exampledb |
| information_schema |
| librarydb |
| mysql |
| performance_schema |
| pharmacy_ecommerce |
| student1db |
| studentdb |
| studentroll_db2 |
| studentrolldb |
| studentrolldb1 |
| students |
| sys |
+-----+

mysql> CREATE DATABASE employeebd_5;
Query OK, 1 row affected (0.03 sec)

mysql> USE employeebd_5;
ERROR 1049 (42000): Unknown database 'employeebd_5'
mysql> USE employeebd_5;
Database changed
mysql> CREATE TABLE IF NOT EXISTS employees (
  ->   emp_id INT PRIMARY KEY,
  ->   emp_name VARCHAR(50),
  ->   dept_id INT,
  ->   salary DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT IGNORE INTO employees VALUES
  -> (101, 'Rahul', 1, 50000),
  -> (102, 'Priya', 2, 65000),
  -> (103, 'Anil', 1, 55000),
  -> (104, 'Sneha', 3, 70000),
  -> (105, 'Kiran', 2, 48000);
Query OK, 5 rows affected (0.04 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM employees;
+-----+-----+-----+-----+
| emp_id | emp_name | dept_id | salary |
+-----+-----+-----+-----+
| 101 | Rahul | 1 | 50000.00 |
| 102 | Priya | 2 | 65000.00 |
| 103 | Anil | 1 | 55000.00 |
| 104 | Sneha | 3 | 70000.00 |
| 105 | Kiran | 2 | 48000.00 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> DROP PROCEDURE IF EXISTS EmployeesBySalary;
Query OK, 0 rows affected, 1 warning (0.04 sec)

mysql>
mysql> DELIMITER $$
```

```

5 rows in set (0.00 sec)

mysql> DROP PROCEDURE IF EXISTS EmployeesBySalary;
Query OK, 0 rows affected, 1 warning (0.04 sec)

mysql>
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE EmployeesBySalary(IN min_salary DECIMAL(10,2))
-> BEGIN
->     SELECT *
->     FROM employees
->     WHERE salary > min_salary;
-> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL EmployeesBySalary(60000);
+-----+-----+-----+-----+
| emp_id | emp_name | dept_id | salary  |
+-----+-----+-----+-----+
|    102 | Priya    |      2  | 65000.00 |
|    104 | Sneha    |      3  | 70000.00 |
+-----+-----+-----+-----+
2 rows in set (0.02 sec)

```

TASK 2: Create a procedure that accepts an employee ID and displays that employee's data

```

mysql> DROP PROCEDURE IF EXISTS UpdateDepartment;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql>
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE UpdateDepartment(
->     IN eid INT,
->     IN new_dept_id INT
-> )
-> BEGIN
->     UPDATE employees
->     SET dept_id = new_dept_id
->     WHERE emp_id = eid;
-> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL UpdateDepartment(101,3);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> SELECT * FROM employees
-> WHERE emp_id = 101;
+-----+-----+-----+-----+
| emp_id | emp_name | dept_id | salary  |
+-----+-----+-----+-----+
|    101 | Rahul    |      3  | 50000.00 |
+-----+-----+-----+-----+
1 row in set (0.02 sec)

```

Create a procedure that accepts employee ID and amount, then increases the employee's salary

```
mysql> DROP PROCEDURE IF EXISTS IncreaseSalary;
Query OK, 0 rows affected, 1 warning (0.02 sec)

mysql>
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE IncreaseSalary(
  ->     IN eid INT,
  ->     IN amount DECIMAL(10,2)
  -> )
  -> BEGIN
  ->     UPDATE employees
  ->     SET salary = salary + amount
  ->     WHERE emp_id = eid;
  -> END $$
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL IncreaseSalary(101,5000);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> SELECT * FROM employees
  -> WHERE emp_id = 101;
+-----+-----+-----+-----+
| emp_id | emp_name | dept_id | salary |
+-----+-----+-----+-----+
|    101 | Rahul    |        3 | 53000.00 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Procedure 4 – Aggregate Function

Create a procedure to calculate the average salary of all employees:

```
mysql> DROP PROCEDURE IF EXISTS EmployeeDepartment;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql>
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE EmployeeDepartment()
-> BEGIN
->     SELECT e.emp_name, d.dept_name, e.salary
->     FROM employees e
->     JOIN departments d ON e.dept_id=d.dept_id;
-> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL EmployeeDepartment();
+-----+-----+-----+
| emp_name | dept_name | salary |
+-----+-----+-----+
| Rahul    | EEE       | 53000.00 |
| Priya    | ECE       | 67000.00 |
| Anil     | CSE       | 55000.00 |
| Sneha    | EEE       | 70000.00 |
| Kiran    | ECE       | 48000.00 |
+-----+-----+-----+
5 rows in set (0.02 sec)
```

Procedure 6 – GROUP BY and AVG

Create a procedure to display department-wise employee count and average salary.

```
mysql> DROP PROCEDURE IF EXISTS DepartmentAverageSalary;
Query OK, 0 rows affected, 1 warning (0.02 sec)

mysql>
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE DepartmentAverageSalary()
-> BEGIN
->     SELECT d.dept_name,
->     COUNT(e.emp_id) AS EmployeeCount,
->     AVG(e.salary) AS AverageSalary
->     FROM employees e
->     JOIN departments d ON e.dept_id=d.dept_id
->     GROUP BY d.dept_id, d.dept_name;
-> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL DepartmentAverageSalary();
+-----+-----+-----+
| dept_name | EmployeeCount | AverageSalary |
+-----+-----+-----+
| EEE       | 2             | 61500.000000 |
| ECE       | 2             | 57500.000000 |
| CSE       | 1             | 55000.000000 |
+-----+-----+-----+
```

Procedure 7 – IF...ELSE

Display High Salary if salary is at least 60000; otherwise display Low Salary.

```
mysql>
mysql> CREATE PROCEDURE SalaryStatus(IN eid INT)
-> BEGIN
->     DECLARE sal DECIMAL(10,2);
->
->     SELECT salary INTO sal
->     FROM employees
->     WHERE emp_id=eid;
->
->     IF sal >= 60000 THEN
->         SELECT 'High Salary' AS Status;
->     ELSE
->         SELECT 'Low Salary' AS Status;
->     END IF;
-> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL SalaryStatus(102);
+-----+
| Status      |
+-----+
| High Salary |
```

Procedure 8 – OUT Parameter

Return the total number of employees using an OUT parameter:

```
mysql>
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE EmployeeCount(OUT total INT)
-> BEGIN
->     SELECT COUNT(*) INTO total
->     FROM employees;
-> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL EmployeeCount(@total);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> SELECT @total AS TotalEmployees;
+-----+
| TotalEmployees |
+-----+
|                5 |
+-----+
1 row in set (0.00 sec)
```

Procedure 9 – INOUT Parameter

Receive a bonus through an INOUT parameter and add 5000 to it:

```
mysql>
mysql> CREATE PROCEDURE AddBonus(INOUT bonus DECIMAL(10,2))
-> BEGIN
->     SET bonus = bonus + 5000;
-> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> SET @bonus=10000;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL AddBonus(@bonus);
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT @bonus AS UpdatedBonus;
+-----+
| UpdatedBonus |
+-----+
|      15000.00 |
+-----+
1 row in set (0.00 sec)
```

Procedure 10 – Transaction

Transfer an amount from one employee salary record to another using STAR TRANSACTION and COMMIT.

```
-- IN amount DECIMAL(10,2)
-- )
-- BEGIN
--     START TRANSACTION;
--
--     UPDATE employees
--     SET salary=salary-amount
--     WHERE emp_id=fromEmp;
--
--     UPDATE employees
--     SET salary=salary+amount
--     WHERE emp_id=toEmp;
--
--     COMMIT;
-- END $$
y OK, 0 rows affected (0.01 sec)

l>
l> DELIMITER ;
l>
l> CALL TransferSalaryAmount(101,102,2000);
y OK, 0 rows affected (0.01 sec)

l> SELECT * FROM employees WHERE emp_id IN (101,102);
+-----+-----+-----+-----+
| emp_id | emp_name | dept_id | salary |
+-----+-----+-----+-----+
| 101    | Rahul    | 3       | 51000.00 |
| 102    | Priya    | 2       | 69000.00 |
+-----+-----+-----+-----+
```

